



Universidad Nacional Autónoma
de México



Facultad de Ciencias

Ingeniería de Software

Documento Evolutivo del Sistema PetConnect

Iteración 3
Versión del documento 3.0

Integrantes:

- De La Rosa Cayetano Caelus - 320033168
- Garcia Ruiz Angel Isaac - 320139198
- Hernández Flores Karina - 317346404
- Palomo Valle Samantha Charlize - 319273841
- Reyes Arteaga Angel David - 319157860
- Rodríguez Jiménez Brenda - 424143273

Profesor:

- Profesora: Karla Socorro García Alcántara
- Ayudante: Maitreyi Martínez Jiménez
- Ayudante de Laboratorio: Luis Gerardo Bernabe Gomez

Semestre: 2026-2

Fecha de actualización: 8 de Mayo de 2026

1 Introducción

El presente documento describe el estado actual del sistema PetConnect en su Iteración 3. En esta fase del proyecto, el desarrollo evolucionó de un sistema de autenticación y gestión de usuarios hacia una plataforma funcional orientada al dominio principal de la aplicación: la adopción de mascotas. Durante esta iteración se incorporó la entidad Animalito como eje central del desarrollo, permitiendo a los usuarios registrados publicar mascotas disponibles para adopción, consultar el listado general de animalitos, visualizar el perfil detallado de cada uno y explorar su ubicación aproximada a través de una vista de mapa geográfico. Adicionalmente, se implementó un mecanismo de comunicación entre usuarios interesados en adoptar y los dueños de los animalitos registrados, mediante el envío de correos electrónicos a través de la plataforma. El objetivo de esta iteración es consolidar el flujo funcional de gestión de animalitos, asegurando la correcta integración entre el frontend y los nuevos endpoints del backend, así como garantizar que las funcionalidades implementadas sean coherentes con los casos de uso definidos para esta fase. Este documento tiene como propósito reflejar de manera clara y coherente las funcionalidades implementadas, los cambios en la arquitectura del sistema y las decisiones tecnológicas adoptadas durante la Iteración 3, manteniendo alineación entre lo desarrollado y lo documentado.

1.1 Alcance de la Iteración 3

Durante la **Iteración 3**, el desarrollo del sistema se centró en las funcionalidades relacionadas con la entidad *Animalito*, así como en la comunicación entre usuarios dentro de la plataforma.

En esta fase, el sistema permite realizar los siguientes procesos:

- **Registro de un animalito:** Proceso disponible exclusivamente para usuarios autenticados.
- **Consulta del listado general:** Visualización de animalitos disponibles para adopción.
- **Consulta del perfil detallado:** Acceso a la información mediante el identificador único del animalito.
- **Visualización en mapa:** Representación de los registros sobre un mapa geográfico con base en su ubicación aproximada.
- **Envío de correo electrónico:** Notificación al dueño del animalito como primer paso formal del proceso de adopción.

1.2 Funcionalidades de la Iteración 3

Las funcionalidades implementadas en esta iteración son las siguientes:

- **Registro de animalito:** Permite a un usuario autenticado publicar una mascota disponible para adopción, proporcionando su información básica. Las coordenadas geográficas son resueltas automáticamente por el sistema a partir del código postal del animalito mediante *Geocodio*, ignorando cualquier coordenada enviada por el cliente.
- **Consulta del listado de animalitos:** Permite visualizar todas las mascotas registradas en la plataforma. Este *endpoint* es público y no requiere autenticación.

- **Consulta del perfil de un animalito:** Permite acceder a la información detallada de una mascota específica mediante su identificador único.
- **Consulta de animalitos cercanos:** Permite al usuario autenticado obtener los animalitos registrados en un radio aproximado de 30 km alrededor de su ubicación, determinada a partir de las coordenadas almacenadas en su perfil.
- **Geolocalización por código postal:** Al registrarse un usuario o al publicar un animalito, el sistema convierte automáticamente el código postal en coordenadas geográficas mediante la integración con *Geocodio*, las cuales quedan almacenadas en el registro correspondiente.
- **Consulta de ubicación del usuario:** Permite al usuario autenticado obtener sus coordenadas geográficas actuales a través del *endpoint* `/usuarios/me/ubicacion`.

1.3 Vistas del *Frontend*

En esta iteración se implementaron y actualizaron las siguientes vistas en el *frontend*:

- **RegisterAnimalito:** Permite a un usuario autenticado registrar una nueva mascota en la plataforma.
- **Home (actualizada):** Muestra el listado de animalitos disponibles. Los filtros por especie están visualmente presentes aunque su integración funcional está pendiente.
- **TarjetaMascota:** Componente reutilizable que presenta la información resumida de cada animalito en el *grid* principal.
- **MascotaPerfil:** Muestra el perfil detallado de un animalito e incluye el flujo de solicitud de adopción, cuyo envío de correo está pendiente de integración con el *backend*.
- **MapaAnimalitos:** Vista implementada en el *frontend* que muestra los animalitos cercanos al usuario sobre un mapa geográfico interactivo, consumiendo el *endpoint* `/animalitos/cercanos` del *backend*.

1.4 Limitaciones Actuales

A pesar de los avances logrados en esta iteración, el sistema aún presenta diversas limitaciones que serán atendidas en fases posteriores:

- La geolocalización depende del servicio externo *Geocodio*. Si dicho servicio no está disponible o el código postal no es reconocido, el sistema lanzará una excepción y el registro fallará. No se contempla un mecanismo de respaldo ante esta situación.
- La comunicación entre usuarios mediante correo electrónico es funcional en el *frontend* y se integra con el servicio *sandbox* de *Mailtrap* en el *backend*. Sin embargo, al operar en entorno *sandbox*, los correos no llegan a destinatarios reales y únicamente pueden consultarse desde el panel de *Mailtrap*. La implementación en producción con un servicio de correo real queda pendiente para iteraciones futuras.
- La carga de fotografías no admite archivos directos desde el dispositivo del usuario; actualmente se solicita una URL externa como referencia de imagen.

- El sistema no contempla la gestión del estado del animalito, por lo que no es posible marcar una mascota como adoptada ni retirar una publicación desde la interfaz.
- Los filtros por especie en la pantalla principal (Todos, Perros, Gatos, Otros) están visualmente presentes pero no están conectados a la lógica de filtrado del arreglo de mascotas.
- El servicio de *Geocodio* opera bajo una cuota diaria de 400 conversiones de código postal por clave de API compartida por el equipo. Dado que cada registro de usuario y cada registro de animalito consume una conversión, se debe evitar realizar pruebas masivas o registros innecesarios para no agotar el límite disponible durante el día.
- El servicio de correo electrónico opera sobre el entorno *sandbox* de *Mailtrap*, el cual impone un límite aproximado de 50 correos por mes compartidos por el equipo. El envío de correos de prueba debe realizarse de forma controlada para no agotar la cuota disponible.

2 Actores del Sistema

Para esta iteración, interactúan los siguientes actores:

- **Usuario no registrado (Visitante):** Persona que accede a la plataforma con el objetivo de crear una cuenta nueva.
- **Usuario registrado (Actor principal):** Cualquier persona que ha validado sus credenciales, tiene una sesión activa y puede acceder a las funcionalidades base del sistema, incluyendo el registro y consulta de animalitos.

3 Requerimientos Funcionales

Los requerimientos funcionales de las iteraciones anteriores se mantienen vigentes. En esta iteración se incorporan los siguientes:

- RF1: Registro de usuarios.** El sistema deberá permitir a los visitantes crear una cuenta proporcionando nombre, correo electrónico, contraseña y código postal.
- RF2: Autenticación de usuarios.** El sistema permitirá el inicio de sesión validando las credenciales criptográficas del usuario y generando un *token* de acceso.
- RF3: Consulta de perfil.** El sistema entregará los datos del usuario autenticado al recibir una petición con un *token* válido.
- RF4: Cierre de sesión.** El sistema permitirá invalidar de manera segura la sesión activa del usuario.
- RF5: Actualización de la información del usuario.** El sistema permitirá al usuario autenticado modificar su información personal.
- RF6: Geolocalización por código postal.** El sistema deberá convertir automáticamente el código postal de un usuario o animalito en coordenadas geográficas (latitud y longitud) mediante el servicio externo *Geocodio*, y almacenarlas en el registro correspondiente.

- RF7: Registro de animalito.** El sistema deberá permitir a un usuario autenticado registrar una mascota disponible para adopción, proporcionando nombre, edad, especie, sexo, tamaño y código postal como campos obligatorios. Las coordenadas geográficas serán resueltas automáticamente por el sistema a partir del código postal.
- RF8: Consulta del listado de animalitos.** El sistema deberá retornar la lista completa de animalitos registrados. Este *endpoint* no requiere autenticación.
- RF9: Consulta del perfil de un animalito.** El sistema deberá permitir consultar la información detallada de un animalito mediante su identificador único.
- RF10: Consulta de animalitos cercanos.** El sistema deberá retornar los animalitos ubicados dentro de un radio aproximado de 30 km alrededor de la ubicación del usuario autenticado, utilizando un algoritmo de *bounding box* sobre las coordenadas almacenadas.
- RF11: Consulta de ubicación del usuario.** El sistema deberá permitir al usuario autenticado obtener sus coordenadas geográficas actuales.

4 Casos de Uso

4.1 CU5: Registro de animalito

Actores involucrados: Usuario registrado.

Descripción breve: Permite a un usuario autenticado publicar una mascota disponible para adopción en la plataforma.

Precondiciones: El usuario debe poseer un *token* de sesión vigente.

Flujo principal:

1. El usuario completa el formulario de registro con los datos de la mascota: nombre, edad, especie, raza (opcional), sexo, tamaño, descripción (opcional), código postal y URL de foto (opcional).
2. El *frontend* envía una petición POST a `/animalitos` incluyendo el *token* en el encabezado `Authorization`.
3. El *backend* valida la presencia del *token* y verifica que los campos obligatorios no estén vacíos.
4. El sistema autentica al usuario localizando su registro mediante el *token* proporcionado.
5. El sistema invoca a `GeocodioService` con el código postal del animalito para obtener sus coordenadas geográficas. Dado que el formulario de registro no expone campos de latitud y longitud al usuario, las coordenadas resueltas por *Geocodio* son el único valor almacenado en la base de datos.
6. El sistema persiste el animalito en la base de datos asociándolo al usuario autenticado.
7. El sistema retorna HTTP 200 con los datos del animalito registrado, incluyendo su identificador único.
8. El *frontend* redirige al usuario a la pantalla principal.

Flujos alternativos:

- 2a. Token ausente:** El *backend* retorna HTTP 401. El *frontend* redirige al *login*.
- 3a. Campos obligatorios faltantes:** El *backend* retorna HTTP 400 con el mensaje “Faltan campos obligatorios”.
- 5a. Código postal no reconocido:** El servicio lanza una excepción en tiempo de ejecución y el registro falla con HTTP 500.

Postcondiciones: El animalito queda registrado en el sistema con coordenadas geográficas resueltas automáticamente y asociado al usuario que lo publicó.

4.2 CU6: Consulta del listado de animalitos

Actores involucrados: Usuario registrado.

Descripción breve: Permite visualizar todos los animalitos disponibles para adopción registrados en la plataforma.

Precondiciones: Ninguna. El *endpoint* GET `/animalitos` es público y no requiere autenticación.

Flujo principal:

1. El *frontend* monta el componente `Home` y ejecuta una petición GET a `/animalitos`.
2. El *backend* consulta la base de datos y retorna la lista completa de animalitos registrados.
3. El *frontend* almacena la lista en el estado del componente y renderiza una `TarjetaMascota` por cada animalito.
4. El usuario puede visualizar la información resumida de cada mascota: nombre, raza, sexo, código postal y edad.

Flujos alternativos:

- 2a. No existen registros:** El sistema retorna una lista vacía y el *frontend* muestra el contador con cero mascotas.

Postcondiciones: El usuario visualiza el listado actualizado de mascotas disponibles para adopción.

4.3 CU7: Consulta del perfil de un animalito

Actores involucrados: Usuario registrado.

Descripción breve: Permite al usuario ver la información completa de una mascota específica y expresar su interés en adoptarla.

Precondiciones: El usuario posee un *token* de sesión vigente. La ruta `/mascota/:id` está protegida por `ProtectedRoute`.

Flujo principal:

1. El usuario hace clic en el botón “Ver perfil” de una `TarjetaMascota`.
2. El *frontend* navega a la ruta `/mascota/:id` y ejecuta una petición GET a `/animalitos/:id`.

3. El *backend* busca el animalito por su identificador y lo retorna.
4. El *frontend* renderiza la vista `MascotaPerfil` con todos los datos del animalito: nombre, especie, sexo, tamaño, edad, raza, código postal, descripción y foto.
5. El usuario puede hacer clic en la foto para verla en un modal de galería.
6. El usuario puede hacer clic en “Me interesa” para iniciar el flujo de solicitud de adopción.
7. Se muestra un modal de confirmación. Al confirmar, el sistema muestra una alerta informativa (envío de correo pendiente de implementación).

Flujos alternativos:

3a. Animalito no encontrado: El *backend* retorna HTTP 404.

3b. Sesión expirada: El *backend* retorna HTTP 401. El *frontend* elimina el *token* y redirige al *login*.

Postcondiciones: El usuario visualiza la información completa del animalito seleccionado.

4.4 CU8: Consulta de animalitos cercanos

Actores involucrados: Usuario registrado.

Descripción breve: Permite al usuario autenticado obtener los animalitos registrados en un radio aproximado de 30 km alrededor de su ubicación geográfica.

Precondiciones: El usuario posee un *token* de sesión vigente y tiene coordenadas geográficas almacenadas en su perfil (derivadas de su código postal al momento del registro).

Flujo principal:

1. El cliente envía una petición GET a `/animalitos/cercanos` incluyendo el *token* en el encabezado `Authorization`.
2. El *backend* autentica al usuario localizando su registro mediante el *token*.
3. El sistema recupera las coordenadas de latitud y longitud almacenadas en el perfil del usuario.
4. El sistema calcula un *bounding box* de ± 30 km ($\text{delta} = 30/111$ grados) alrededor de las coordenadas del usuario.
5. El sistema ejecuta la consulta JPQL `buscarCercanos` sobre `AnimalitoRepository`, filtrando los animalitos cuyas coordenadas se encuentren dentro del *bounding box* calculado.
6. El sistema retorna la lista de animalitos encontrados en formato JSON.

Flujos alternativos:

2a. Token inválido: El sistema retorna HTTP 404 (respuesta de *not found*).

3a. Sin coordenadas: El servicio retorna null y el sistema responde con HTTP 404.

6a. Sin resultados: El sistema retorna una lista vacía con HTTP 200.

Postcondiciones: El usuario recibe la lista de animalitos geográficamente cercanos a su ubicación.

5 Tecnologías Utilizadas

5.1 Backend

El *backend* continúa desarrollado en Kotlin con Spring Boot, MariaDB, Spring Data JPA y Maven, tal como se documentó en la iteración anterior. En esta iteración se incorporan las siguientes tecnologías y herramientas adicionales:

Geocodio API. Servicio externo de geocodificación que convierte un código postal en coordenadas geográficas (latitud y longitud). Se integra al sistema mediante el cliente HTTP `RestTemplate` de Spring, realizando una petición `GET` al *endpoint* de la API con el código postal y la clave de acceso como parámetros. Esta integración es utilizada tanto en el registro de usuarios como en el registro de animalitos, garantizando que ambas entidades cuenten con coordenadas geográficas precisas y consistentes.

RestTemplate. Cliente HTTP sincrónico provisto por el *framework* Spring, utilizado para realizar las peticiones al *endpoint* de *Geocodio*. Se configura como un *Bean* de Spring mediante la clase `GeocodioConfig`, lo que permite su inyección de dependencias en `GeocodioService`.

dotenv-kotlin (`io.github.cdimascio:dotenv-kotlin`). Biblioteca que permite cargar variables de entorno desde un archivo `.env` al inicio de la aplicación. Se utiliza para gestionar de forma segura la clave de acceso a la API de *Geocodio*, evitando que dicha información sensible quede expuesta directamente en el código fuente.

5.2 Frontend

El *frontend* continúa desarrollado con React y Vite, utilizando Axios como cliente HTTP y `sessionStorage` para el manejo del *token* de autenticación, tal como se estableció en la iteración anterior. En esta iteración se incorporaron los siguientes componentes y páginas nuevas:

React Router DOM. Continúa siendo la solución de enrutamiento del *frontend*. En esta iteración se registraron dos nuevas rutas protegidas: `/animalitos/register` para el registro de animalitos y `/mascota/:id` para el perfil detallado de un animalito. Ambas rutas están resguardadas por el componente `ProtectedRoute`.

Lucide React. Biblioteca de íconos vectoriales utilizada en los componentes de la interfaz. En esta iteración se incorporan nuevos íconos en la vista `MascotaPerfil`: `Heart`, `MapPin`, `Calendar`, `Tag`, `HouseHeart`, `ShieldCheck`, `X` y `ArrowBigDown`.

6 Arquitectura General

6.1 Backend - API REST en capas

El *backend* mantiene la arquitectura de múltiples capas (*N-Tier Architecture*) establecida en la iteración anterior. En esta iteración se extiende la arquitectura incorporando la entidad *Animalito* con sus capas correspondientes, y se agrega una nueva capa de integración con servicios externos:

Capa de Presentación (*Controllers*): Se incorpora `AnimalitoController`, que expone los *endpoints* REST para la gestión de animalitos: `POST /animalitos` para el registro, `GET /animalitos` para el listado público, `GET /animalitos/{id}` para la consulta por identificador, y `GET /animalitos/cercanos` para la consulta geolocalizada. El `UsuarioController` se extiende con el *endpoint* `GET /usuarios/me/ubicacion`.

Capa de Lógica de Negocio (*Services*): Se incorpora `AnimalitoService`, que orquesta las operaciones de negocio sobre la entidad *Animalito*. Esta capa es responsable de validar el *token* del usuario, invocar a `GeocodioService` para resolver las coordenadas a partir del código postal, y ejecutar las consultas de animalitos cercanos aplicando el algoritmo de *bounding box*. Se agrega `GeocodioService` como un servicio de integración con la API externa de *Geocodio*.

Capa de Acceso a Datos (*Repositories*): Se incorpora `AnimalitoRepository`, que extiende `CrudRepository` y define dos métodos de acceso a datos: `findByUsuarioId` para recuperar los animalitos de un usuario específico, y `buscarCercanos`, una consulta JPQL personalizada que filtra animalitos por rango de coordenadas geográficas.

Capa de Integración Externa (*Services*): `GeocodioService` actúa como adaptador hacia la API externa de *Geocodio*. Recibe un código postal, construye la URL de la petición con la clave de API gestionada mediante `dotenv`, ejecuta la petición HTTP con `RestTemplate` y extrae las coordenadas del JSON de respuesta.

6.2 Frontend - Arquitectura basada en componentes

El *frontend* mantiene la estructura de capas establecida en la iteración anterior. En esta iteración se incorporan nuevos componentes y páginas dentro de las capas existentes:

Api. Se extiende `servicioAnimalito.js` con tres funciones: `registrarAnimalito` (`POST /animalitos`), `obtenerAnimalitos` (`GET /animalitos`) y `obtenerAnimalitoPorId` (`GET /animalitos/:id`).

Componentes. Se incorpora `TarjetaMascota`, un componente reutilizable que renderiza la información resumida de un animalito y permite navegar a su perfil detallado.

Páginas. Se incorporan `RegisterAnimalito`, que gestiona el formulario de registro de una mascota, y `MascotaPerfil`, que presenta la información completa de un animalito e incluye el modal de galería y el flujo de solicitud de adopción.

Estilos. Se agregan los archivos CSS correspondientes a las nuevas páginas y componentes.

6.3 Diagrama de Arquitectura

La siguiente figura muestra la interacción general entre los componentes principales del sistema PetConnect:

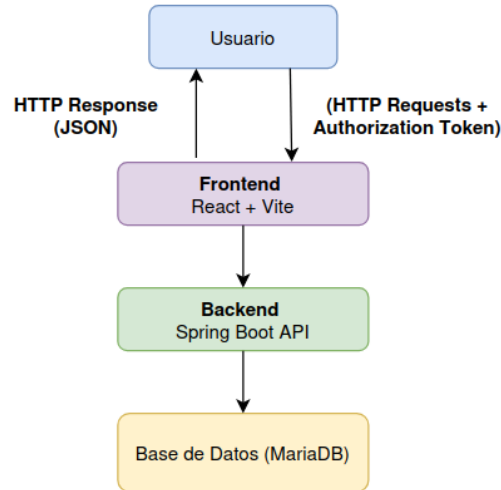


Diagrama 1: Arquitectura general del sistema PetConnect

El diagrama representa la arquitectura general del sistema PetConnect. El usuario interactúa con el frontend, que envía solicitudes HTTP con un token de autorización al backend. El backend procesa la información, consulta la base de datos y devuelve respuestas en formato JSON, las cuales son mostradas nuevamente al usuario a través del frontend.

7 Flujo de Autenticación

El flujo de autenticación se mantiene igual al descrito en la Iteración 2, con la única diferencia de que el proceso de registro ahora incluye el campo código postal como dato obligatorio. Al completarse el registro, el sistema invoca a `GeocodioService` para convertir el código postal en coordenadas geográficas, las cuales quedan almacenadas en el perfil del usuario desde el primer momento.

El proceso de inicio de sesión, manejo del *token* mediante `sessionStorage`, envío automático del *token* en el encabezado `Authorization` a través del interceptor de Axios, la protección de rutas con `ProtectedRoute` y el cierre de sesión permanecen sin cambios respecto a la iteración anterior.

8 Modelo Conceptual de Datos

En esta iteración, el modelo conceptual de datos se extiende para incorporar la entidad *Animalito*, la cual representa a las mascotas publicadas por los usuarios en la plataforma. Adicionalmente, la entidad *Usuario* se actualiza para incluir campos de geolocalización.

8.1 Entidad Usuario (actualizada)

La entidad *Usuario* incorpora tres nuevos atributos respecto a la iteración anterior:

- **Código postal:** Campo obligatorio en el registro. Representa la ubicación general del usuario y sirve como entrada para el proceso de geocodificación.

- **Latitud y Longitud:** Coordenadas geográficas del usuario, derivadas automáticamente de su código postal mediante *Geocodio* al momento del registro. Estos campos son utilizados por el sistema para calcular la proximidad con los animalitos registrados.

8.2 Entidad Animalito (nueva)

La entidad *Animalito* representa a una mascota disponible para adopción. Sus atributos son los siguientes:

- **ID de Animalito:** Clave primaria numérica autoincrementable, gestionada por la base de datos.
- **Nombre, Edad, Especie, Sexo y Tamaño:** Campos obligatorios que describen la identidad básica de la mascota.
- **Raza y Descripción:** Campos opcionales que enriquecen la información de la mascota.
- **Código postal:** Campo obligatorio que indica la ubicación del animalito.
- **Latitud y Longitud aproximadas:** Coordenadas geográficas resueltas automáticamente por *GeocodioService* a partir del código postal del animalito, independientemente de los valores enviados por el cliente.
- **URL de foto:** Campo opcional que almacena la referencia a la imagen de la mascota mediante una URL externa.
- **Marca de tiempo:** Los campos `createdAt`, `updatedAt` y `deletedAt` se gestionan automáticamente mediante los *callbacks* `@PrePersist` y `@PreUpdate` de JPA, manteniéndose la posibilidad de eliminaciones lógicas en futuras iteraciones.

8.3 Relación entre entidades

La entidad *Animalito* mantiene una relación de muchos a uno (`@ManyToOne`) con la entidad *Usuario*: un usuario puede registrar múltiples animalitos, pero cada animalito pertenece a exactamente un usuario. Esta relación se materializa mediante la columna `usuario_id` en la tabla `animalito`, gestionada por JPA a través de la anotación `@JoinColumn`.

9 Requerimientos No Funcionales

Los requerimientos no funcionales de las iteraciones anteriores se mantienen vigentes. En esta iteración se incorporan los siguientes:

- RNF1: Seguridad.** Las contraseñas de los usuarios no deberán almacenarse en texto plano; deberán persistirse mediante el algoritmo *hash* SHA-256.
- RNF2: Persistencia.** La información de usuarios y animalitos deberá almacenarse en una base de datos relacional accesible por el *backend*.

RNF3: Integridad de datos. El sistema deberá validar la presencia de los campos obligatorios antes de procesar cualquier solicitud de registro.

RNF4: Mantenibilidad. El *backend* deberá organizarse en capas separadas de controlador, servicio y repositorio para facilitar su evolución.

RNF5: Gestión de sesiones. El sistema deberá invalidar *tokens* al cerrar sesión.

RNF6: Protección de credenciales externas. Las claves de acceso a servicios externos como *Geocodio* no deberán estar expuestas en el código fuente. Deberán gestionarse mediante variables de entorno cargadas desde un archivo de configuración local.

RNF7: Consistencia geográfica. Las coordenadas geográficas de un animalito deberán ser resueltas siempre por el sistema mediante *Geocodio*, independientemente de los valores enviados por el cliente, garantizando así la integridad de los datos de ubicación.

RNF8: Disponibilidad de servicios externos. El sistema depende de la disponibilidad de la API de *Geocodio* para completar el registro de usuarios y animalitos. En caso de falla del servicio externo, la operación no podrá completarse.

10 Configuración de Variables de Entorno

El sistema requiere la configuración de variables de entorno sensibles para su correcto funcionamiento. Estas variables deben definirse en un archivo `.env` ubicado en la raíz del proyecto *backend* y no deben incluirse en el repositorio de control de versiones. A continuación se describen las variables requeridas y las restricciones de uso asociadas a cada servicio.

10.1 Geocodio - Geolocalización

Geocodio es el servicio externo utilizado para convertir códigos postales en coordenadas geográficas. La clave de API es compartida por todos los integrantes del equipo, por lo que su cuota de uso es acumulativa.

Límite de uso: 400 conversiones de código postal por día, compartidas entre todos los integrantes del equipo. Cada registro de usuario y cada registro de animalito consume una conversión. Se debe evitar realizar registros de prueba innecesarios para no agotar el límite diario disponible.

10.2 Mailtrap - Envío de correos electrónicos

Mailtrap es el servicio utilizado para el envío de correos electrónicos en el entorno de desarrollo (*sandbox*). Los correos enviados desde el *sandbox* no llegan a destinatarios reales; pueden inspeccionarse desde el panel de *Mailtrap*.

Límite de uso: Aproximadamente 50 correos por mes en el plan *sandbox*, compartidos por el equipo. El envío de correos de prueba debe realizarse de forma controlada para no agotar la cuota mensual disponible.

11 Versionamiento

Como parte del cierre de la Iteración 3, se establece el siguiente versionamiento:

- El ***backend*** se encuentra en la versión 3.0.0, incorporando la entidad *Animalito* con sus operaciones de registro y consulta, la integración con el servicio externo *Geocodio* para geocalización automática, el *endpoint* de animalitos cercanos con algoritmo de *bounding box*, y el *endpoint* de ubicación del usuario autenticado.
- El ***frontend*** se establece en la versión 2.0.0, incorporando las vistas de registro de animalito, perfil detallado de animalito y el servicio de comunicación con los nuevos *endpoints* del *backend*.
- Finalmente, **este documento** corresponde a la versión 3.0, ya que refleja la evolución del sistema en esta tercera iteración, incluyendo los cambios en la arquitectura, las nuevas entidades del modelo de datos y las funcionalidades implementadas.